

Programação Orientada a Objetos

Conceitos de Orientação a Objetos



Programação Estruturada

- **Base:**

- Sequência: Uma tarefa é executada após a outra, linearmente.
- Decisão: A partir de um teste lógico, determinado trecho de código é executado, ou não.
- Iteração: A partir de um teste lógico, determinado trecho de código é repetido por um número finito de vezes.

- **Vantagens**

- É fácil de entender. Ainda muito usada em cursos introdutórios de programação.
- Execução mais rápida.

- **Desvantagens**

- Baixa reutilização de código
- Códigos confusos: Dados misturados com comportamento



Programação Orientada a Objetos

- **Base**

- Classes e Objetos
- Métodos e Atributos

- **Vantagens**

- Melhor organização do código
- Bom reaproveitamento de código

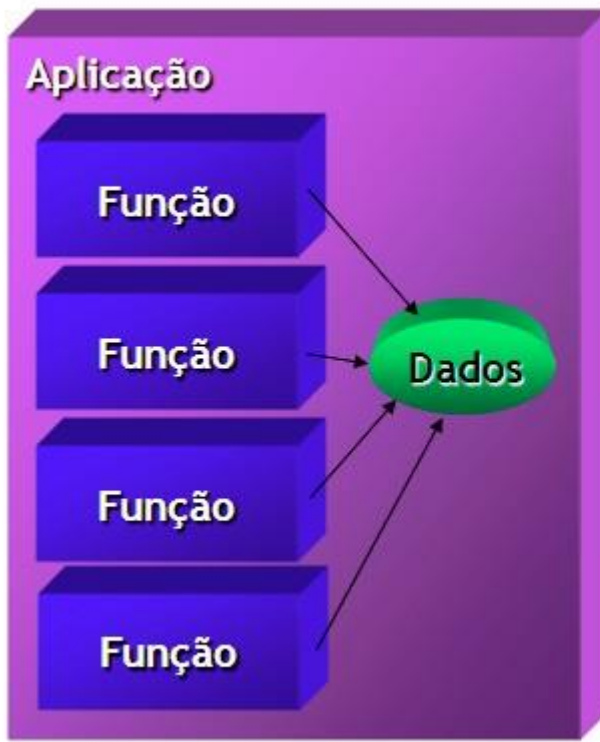
- **Desvantagens**

- Desempenho mais baixo que o paradigma estruturado
- Mais difícil compreensão

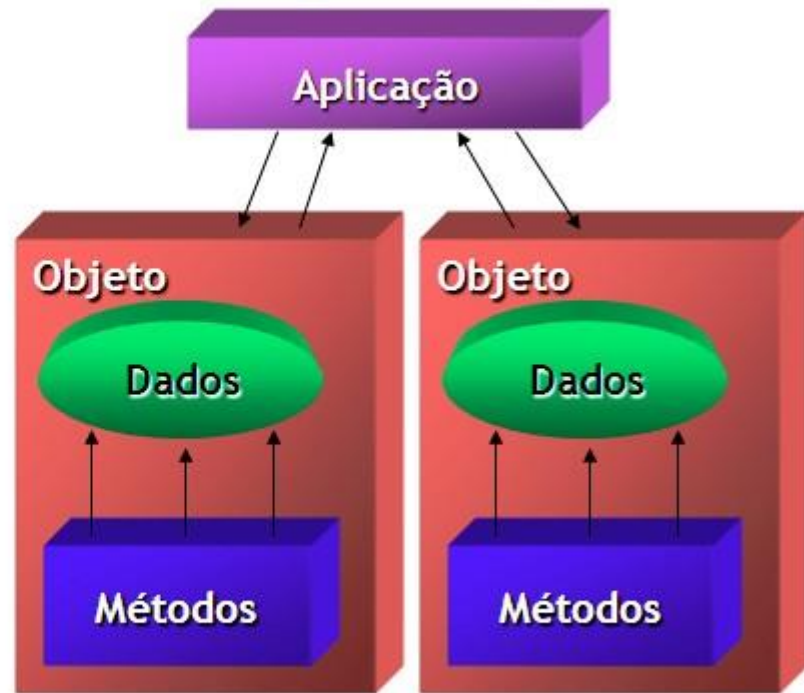


Programação Estruturada vs Orientada a Objetos

Estruturada



Orientação a Objetos



Origem da P.O.O.

- Nos anos 70 surge Smalltalk, a primeira linguagem totalmente em Orientação a Objeto (O.O)
- C++, evolução de C, já possuía conceitos O.O
- Na década de 80 praticamente todas as linguagens já usavam conceitos O.O
- Delphi
- PASCAL
- Java



Conceito da O.O.

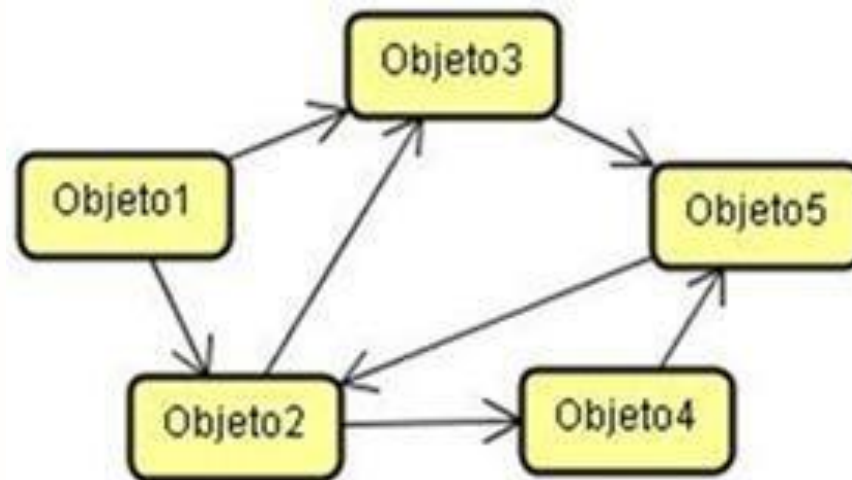
- “Uma nova maneira de pensar os problemas utilizando conceitos do Mundo Real. O componente fundamental é o objeto que combina estrutura e comportamento em uma única entidade”

Raumbaugh

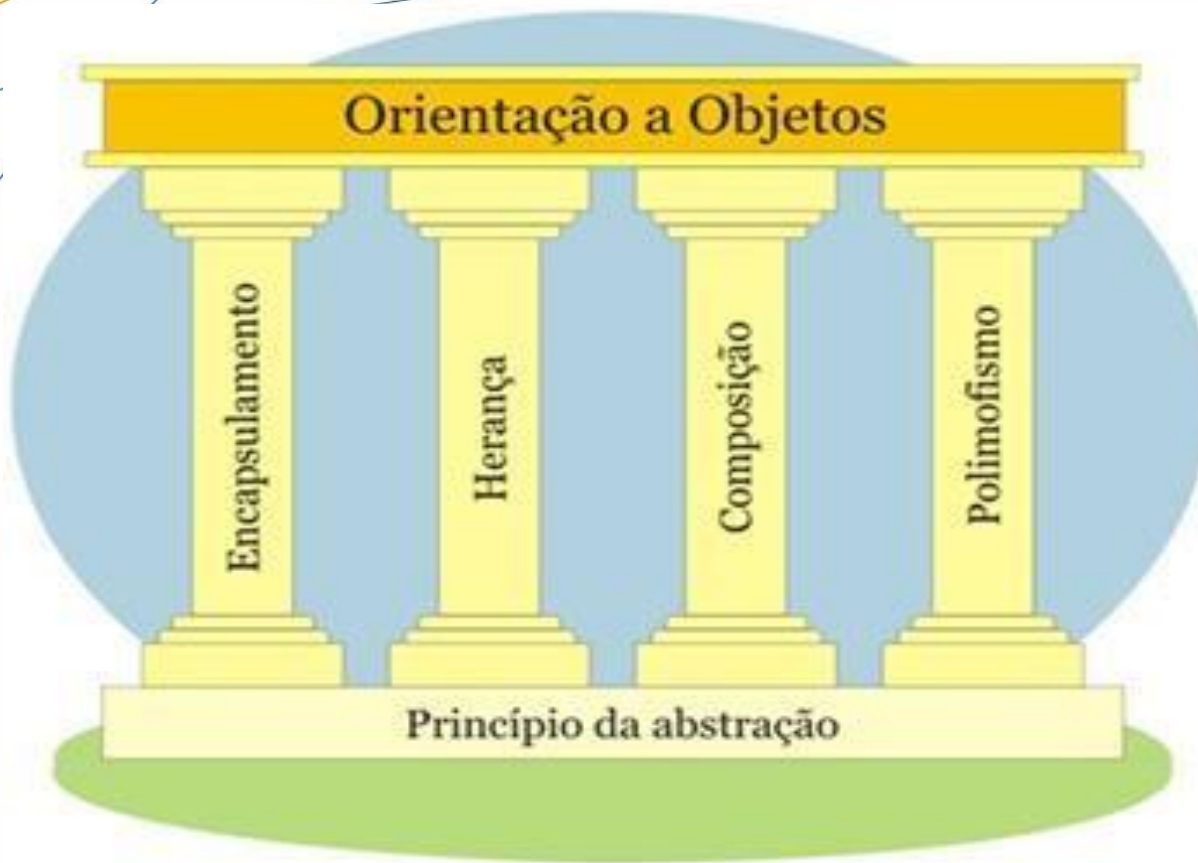
- “Sistema orientado a objetos é uma coleção de objetos que interagem entre si” ***Bertrand Meyer***



Conceito da O.O.



Conceito da O.O.



Conceito da O.O.

- Abstração:
 - Habilidade de se concentrar nos aspectos essenciais do sistema, ou um contexto qualquer, ignorando o que é supérfluo;
 - Abstração é uma operação que consiste em isolar, por exemplo num conceito, um elemento genérico que permita exclusão de outros, dos quais se faz abstração.

Objetos

O que é:

- Representação computacional de algo do mundo real
 - Concreto
 - Abstrato

Abstração

- Transformar aquilo que observamos realidade para a virtualidade
- Cada objeto possui um conjunto de **características** e **comportamentos**



Objetos

Concretos

- Cão
- Moto
- Casa

Abstratos

- Música
- Transação Bancária

Modelo

- Características + Comportamento

Objetos

Estado

- Atributos (Características)

Operações

- Métodos (Comportamentos)

Identidade

- Dois objetos com estado e operações precisamente idênticos não são iguais

Operações podem mudar os valores dos atributos assim mudando o estado de um objeto.



Objetos

Uma **característica** descreve uma propriedade de um objeto, ou seja, algum elemento que descreva o objeto

Exemplo de características de um objeto identificado como carro:

Cor Marca

Número de portas

Ano de fabricação e tipo de combustível



Objetos

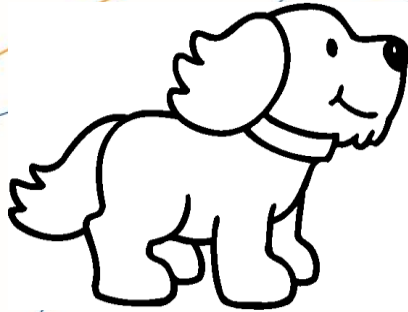
Um **comportamento** representa uma ação ou resposta de um objeto a uma ação do mundo real.

Exemplos de comportamento para o objeto carro

Acelerar Parar Andar Estacionar



Métodos e Atributos



- Atributos

- Raça: Poodle
- Nome: Rex
- Peso: 5 quilos

- Métodos

- Latir
- Comer
- Dormir



- Potência: 500cc
- Modelo: Honda
- Ano: 1998

- Acelerar
- Frear
- Abastecer

Identificação

- Identificar as características e o comportamento de objetos do mundo real é o primeiro passo da programação OO.
- Observe um objeto e pergunte:
 1. Quais os possíveis características deste objeto e quais estados elas assumem?
 2. Quais comportamentos (ações) que ele pode executar?

Classe

- Conjunto de objetos:
 - Características semelhantes
 - Comportamento comum
 - Interação com outros objetos
- Uma classe é a forma para criação de objetos
- Objetos são representações concretas (instâncias) de uma classe
- Objetos com características e comportamentos semelhantes são agrupados em classes.

Métodos e Atributos



Gato

New



Gato

Raça: Savannah

Nome: Gatuno Peso:

2,5 kg

Idade: 2 anos

New



Gato

Raça: Maine Moon

Nome: Listrado Peso:

g

Idade: 5 anos

New



Gato

Raça: Siamês Nome:

Bichano Peso: 4 kg

Idade: 3 anos

Classe vs Objeto

Classe

Cliente
-idCliente : Long
-cpf : String
-nomeCliente : String
-endereço : String
-dtNascimento : Date
-renda : Decimal
+obter() : Boolean
+salvar() : Boolean
+excluir() : Boolean
+possuiDebito() : Boolean

Instâncias (Objetos)

Pedro Afonso : Cliente

idCliente : Long = 1
cpf : String = 111.111.111-11
nomeCliente : String = Pedro Afonso
endereço : String = Rua 1 nº 2 Centro
dtNascimento : Date = 01/01/1970
renda : Decimal = 1.200,00

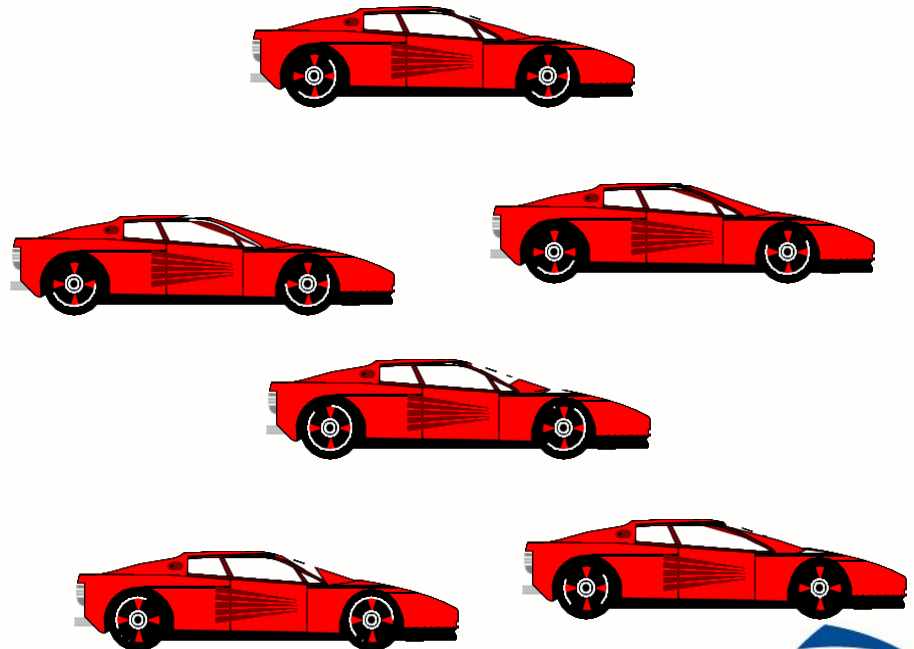
Maria Souza : Cliente

idCliente : Long = 2
cpf : String = 222.222.222-22
nomeCliente : String = Maria Souza
endereço : String = Av Independência nº 1425
dtNascimento : Date = 10/04/1980
renda : Decimal = 4.500,00

Objetos (Instâncias) da classe Cliente

Classe vs Objeto

Carro
Número de Rodas Cor Cor Lateral
Anda Para Acelera Estaciona



Instâncias da Classe

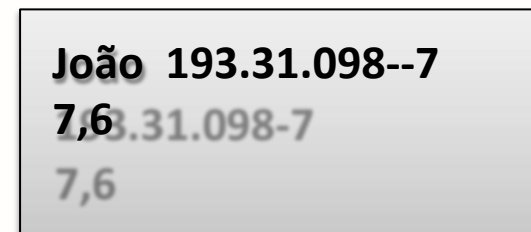
- As classes proveem a estrutura para a construção de objetos -- estes são ditos **instâncias** das classes

Classe

Instâncias

atributos

métodos



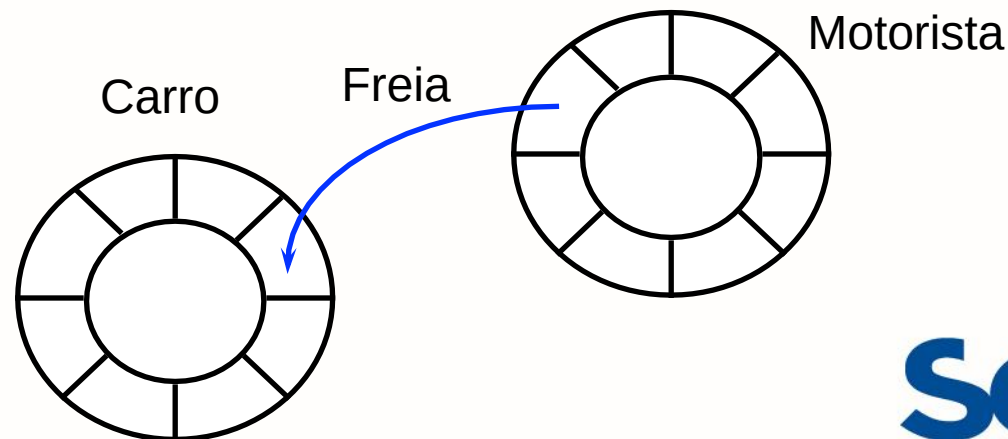
Instâncias da Classe

- Um objeto é uma instância de uma única classe.
- Uma instância de um objeto é uma unidade de programação que é armazenada em uma variável.
- Um programa orientado a objetos é composto por um conjunto de objetos que interagem entre si.



Comunicação de objetos

- Um programa OO é **um conjunto de objetos que colaboram entre si** para a solução de um problema
- Objetos **colaboram através de trocas de mensagens**
- A troca de mensagem representa a chamada de um método



Comunicação de objetos

- Um **envio de mensagem** sempre possui:
 - Um emissor
 - Um receptor
 - Um seletor de mensagens (nome do método chamado)
 - Parâmetros (opcionais)
- Uma mensagem **pode retornar um valor**



Comunicação de objetos

- Métodos operam no estado interno de um objeto e servem como mecanismo de comunicação entre objetos.



Métodos em classes de Java

```
Qualificador_de_acesso class Nome_Da_Classe
{
    // atributos da classe
    // métodos da classe
}
```

```
// Class Lampada
public class Lampada
{
    // Atributos
    boolean acesa;

    // Métodos
    public void ligar()
    {   acesa = true;   }
    public void desligar()
    {   acesa = false; }
}
```

```
// Class Bicicleta
class Bicicleta
{
    // Atributos
    int velocidade = 0;
    int marcha = 1;

    // Métodos
    void mudarMarcha(int novoValor)
    {   marcha= novoValor;   }
    void aumentaVelocidade(int incremento)
    {   velocidade+= incremento; }
}
```



Atividade

- Liste as classes de um super mercado por exemplo “Guanabara”:



Encapsulamento

Definição

- É a utilização de técnicas de programação e mecanismos de linguagem de programação para **agrupar e restringir acesso à atributos métodos e classes**
- Objetivo:
 - Reduzir a complexidade externa (interface) das classes Preservar a integridade dos dados internos dos objetos

Encapsulamento

- Um objeto, em um programa, “encapsula” todo o seu estado e o comportamento;
- Os **dados e as operações** são agrupados e a sua **implementação é escondida, protegida** dos usuários;



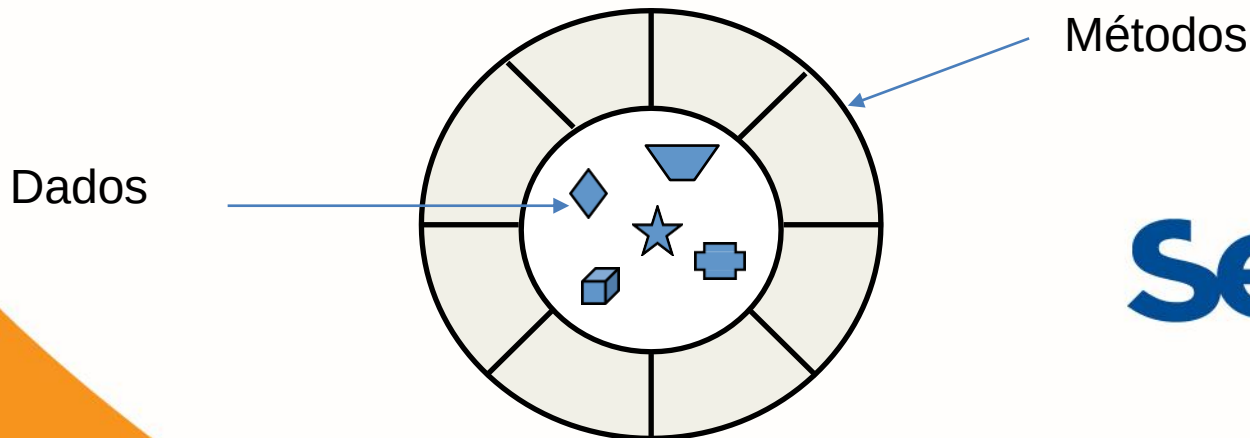
Encapsulamento

- É um princípio fundamental da OO:
- Esconder o estado interno (valores dos atributos).
- Obrigar que interações com os atributos sejam executadas através de métodos.
- Com o encapsulamento um objeto determina a permissão que outros objetos terão para acessar seus atributos (estado).



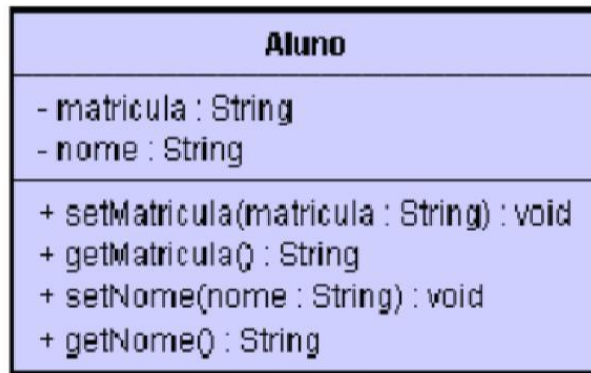
Encapsulamento de Dados

- Atributos e Métodos
 - Os métodos formam uma “cerca” em torno dos atributos. Os atributos não podem ser manipulados diretamente.
 - Os atributos somente podem ser alterados ou consultados através dos métodos do objeto.



Classes em Java

Classe na UML



```
public class Aluno {  
  
    private String matricula;  
    private String nome;  
  
    public void setMatricula(String matricula) {  
        this.matricula = matricula;  
    }  
    public String getMatricula() {  
        return matricula;  
    }  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
    public String getNome() {  
        return nome;  
    }  
}
```

Criando Objetos em Java

Para instanciarmos um novo objeto devemos utilizar o operador `new`, como nos exemplos abaixo:

```
NomeDaClasse nomeDoObjeto = new NomeDaClasse();
```

- Criando dois objetos bicicleta:

```
Bicicleta bicicleta1 = new Bicicleta();
```

```
Bicicleta bicicleta2 = new Bicicleta();
```

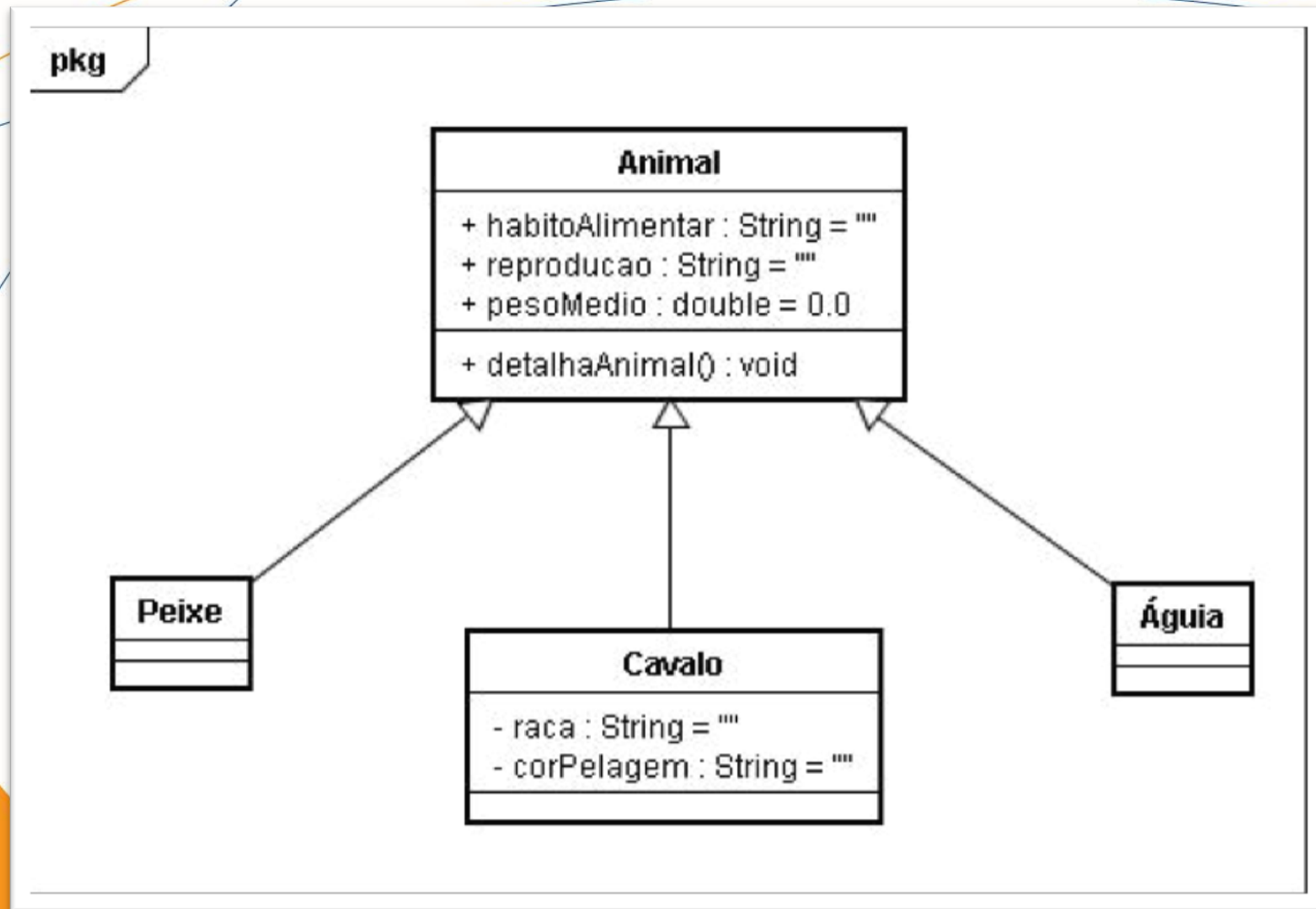
- Invocando seus métodos:

```
bicicleta1.mudarMarcha(2);
```

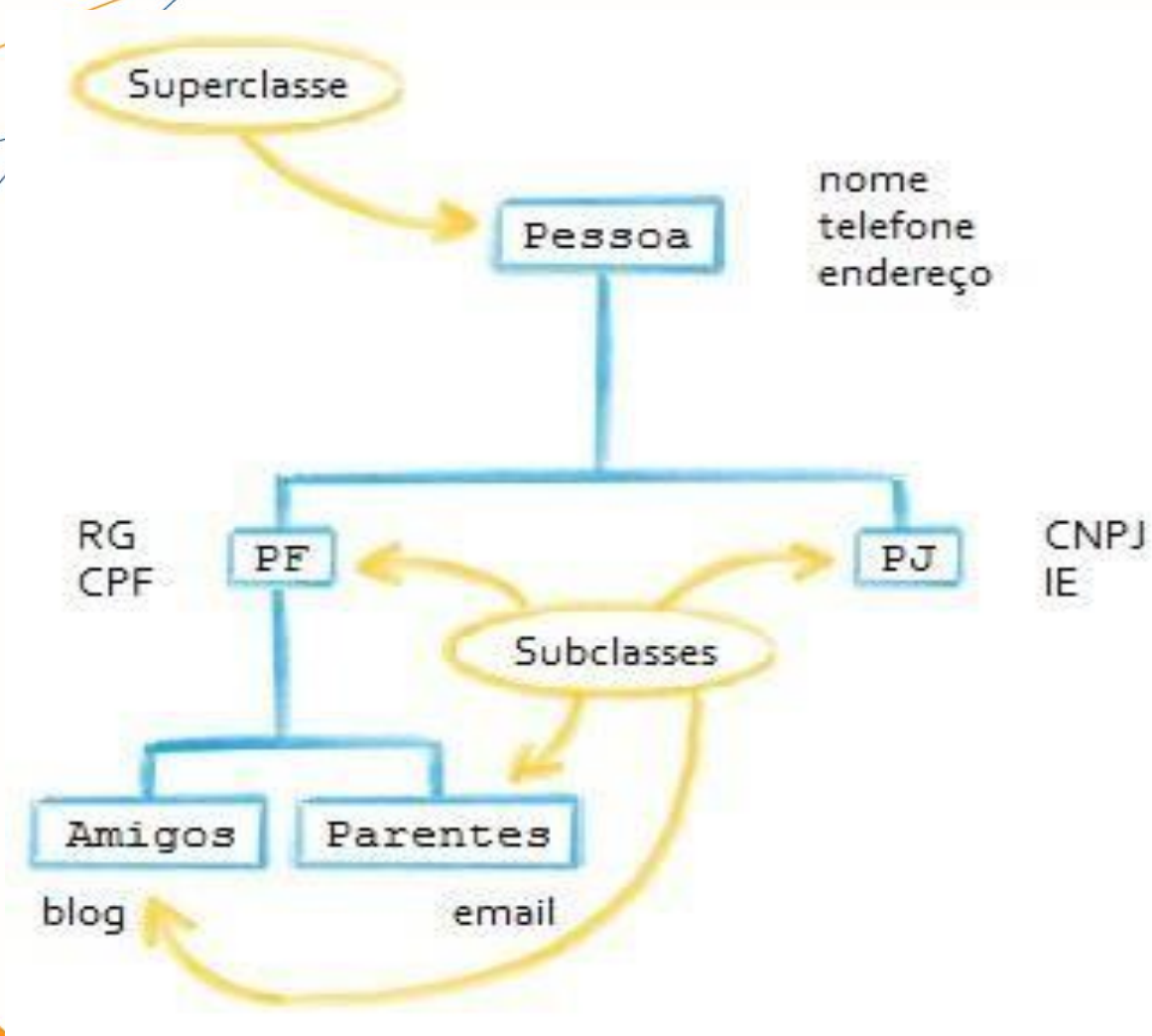
```
bicicleta2.aumentaVelocidade(5);
```



Herança na UML



Herança na UML



Herança na UML

Superclasse

Pessoa

nome
telefone
endereço

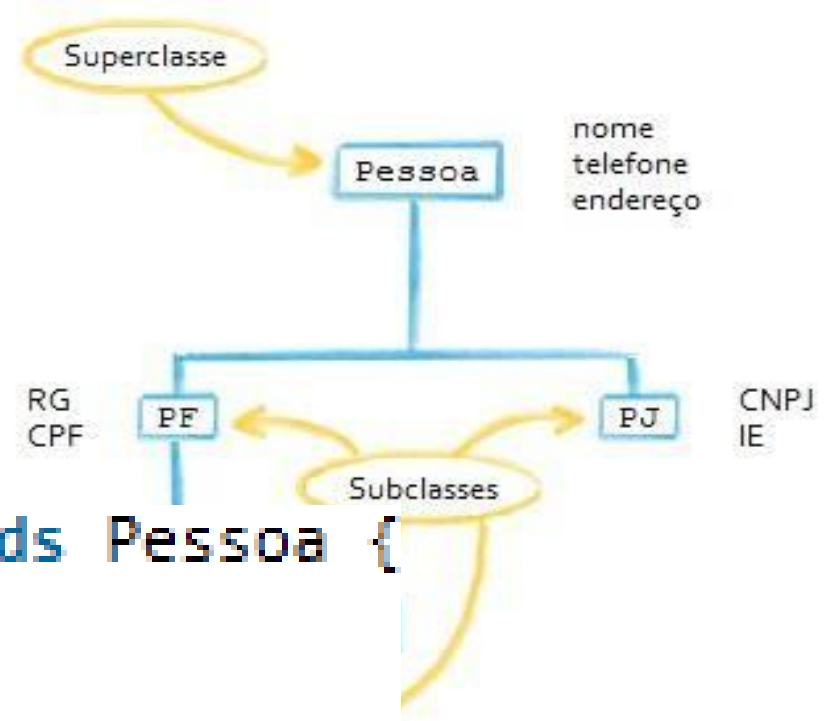
```
01 public class Pessoa {  
02     String nome;  
03     String telefone;  
04     String endereço;  
05 }
```

```
06 public Pessoa() {  
07     nome = "";  
08     telefone = "";  
09     endereço = "";  
10 }
```

Construtor



Herança na UML

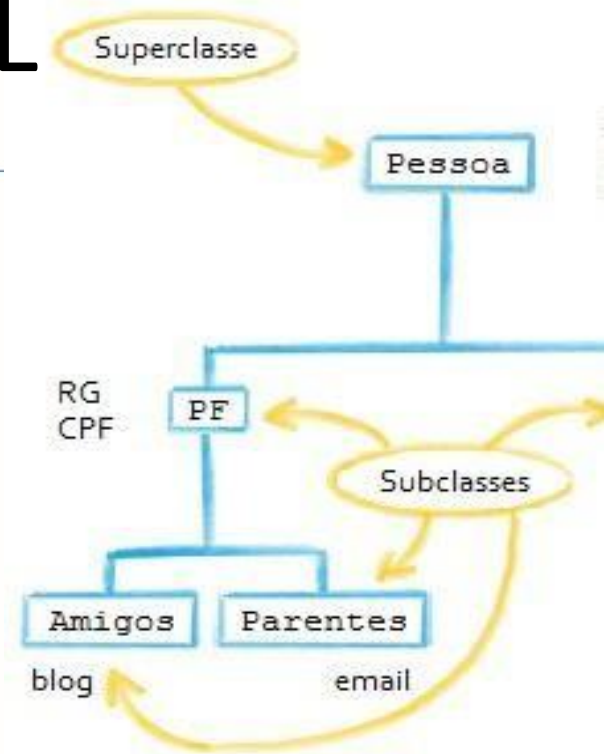


```
01 public class PF extends Pessoa {
02     String RG;
03     String CPF;
04
05     public PF() {
06         super();
07         RG = "";
08         CPF = "";
09     }
```

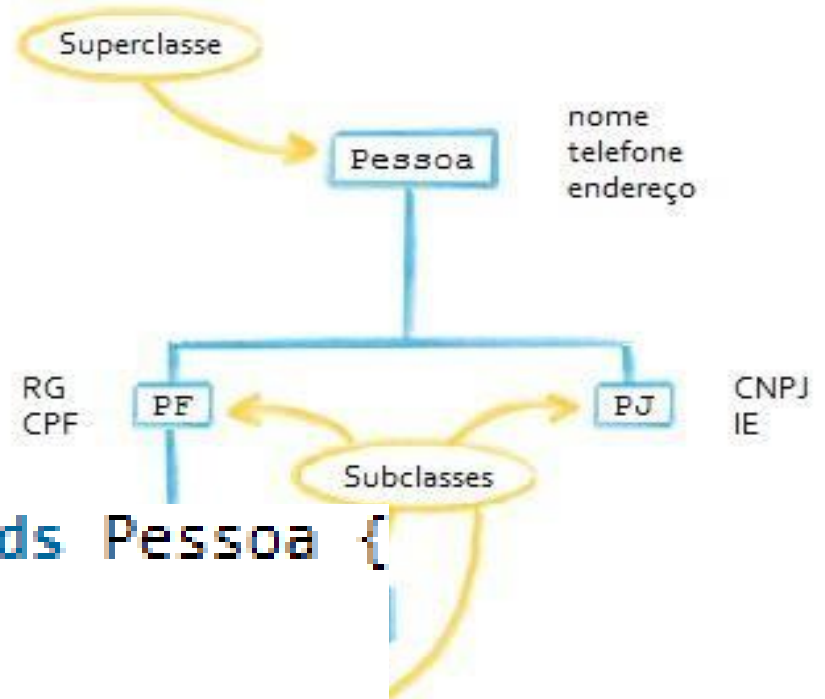
Herança na UML

```
01 public class Amigos extends PF {  
02     String blog;  
03  
04     public Amigos() {  
05         super();  
06         blog = "";  
07     }  
/
```

```
01 public class Parentes extends PF {  
02     String email;  
03  
04     public Parentes() {  
05         super();  
06         email = "";  
07     }  
.
```



Herança na UML



```
01 public class PJ extends Pessoa {
02     String CNPJ;
03     String IE;
04
05     public PJ() {
06         super();
07         CNPJ = "";
08         IE = "";
09     }
```

Visibilidade

- - Private
 - Somente a classe tem acesso Não é transmitido por herança
- ~ Default ou Friendly
 - Acesso a classe inteira
 - Visível para as classes do mesmo pacote
 - Só é transmitido por herança em classes do mesmo pacote

Visibilidade

- **# Protected**
 - Visível em toda a classe
 - Visível em todas as classes de um pacote
 - Transmitido por herança
- **+ Public**
 - Visível irrestritamente

Visibilidade

Modificador	Classe	Subclasse	Pacote	Todos
<u>Public Java</u>	☒	☒	☒	☒
<u>Default Java</u>	☒		☒	
<u>Protected Java</u>	☒	☒	☒	
<u>Private Java</u>	☒			

Atividade

O que vai ser transmitido pela herança?

